



PyQuest

Information- & Aufgabenblatt

Kapitel 5: Operatoren

Rechnen, zuweisen, vergleichen und logisch verknüpfen.

Arithmetische Operatoren

Operatoren sind spezielle Symbole, mit denen du Werte verrechnest oder vergleichst. Die Werte, mit denen ein Operator arbeitet, heißen **Operanden**: Bei $7 + 3$ sind 7 und 3 die Operanden und + der Operator. Die bekanntesten sind die **arithmetischen** Operatoren für die Grundrechenarten:

- + Addition · - Subtraktion · * Multiplikation · / Division

Ein paar Rechnungen – achte auf die Division:

Beispiel

```
print(3 + 5)
print(5.7 - 2)
print(2 * 100.0)
print(6 / 3)
```

Zwei besondere Operatoren solltest du kennen:

- % (Modulo) liefert den **Rest** einer Division: $7 \% 4 \rightarrow 3$ ($7 = 1 \cdot 4 + 3$)
- // (Ganzzahldivision) liefert das ganzzahlige Ergebnis ohne Rest: $7 // 4 \rightarrow 1$

Modulo ist überraschend nützlich – z. B. um zu prüfen, ob eine Zahl **gerade** ist: `zahl % 2 == 0`.

Rechne das **nicht im Kopf** aus – dafür hast du ja Python. Drücke auf „Ausführen“ und schau dir an, was Modulo macht:

Beispiel

```
print(144 % 7)          # Rest bei 144 geteilt durch 7
print(123456 % 100)     # die letzten beiden Ziffern
print(7 % 2)            # 1 -> ungerade
print(8 % 2)            # 0 -> gerade
```



Aufgabe 1: Du hast es gerade gesehen: Woran erkennst du mit %, ob eine Zahl gerade ist?

- ☐ a) $\text{zahl} \% 2$ ergibt 0
- ☐ b) $\text{zahl} \% 2$ ergibt 1
- ☐ c) $\text{zahl} \% 0$ ergibt 2

Aufgabe 2: Prüfe mit Modulo, ob 11 gerade ist: Gib $11 \% 2$ aus. (Bei einer geraden Zahl käme 0 heraus, hier also 1.)

Dein Code:



Zuweisungsoperatoren

Mit **Zuweisungsoperatoren** gibst du einer Variable einen Wert. Der Grundoperator ist das `=`.

Es gibt aber praktische **kombinierte** Zuweisungsoperatoren, die einen Wert verändern und gleich wieder speichern:

- `x += 5` bedeutet `x = x + 5` (dazuzählen)
- `x -= 5` bedeutet `x = x - 5` (abziehen)
- `x *= 5` bedeutet `x = x * 5` (malnehmen)
- `x /= 5` bedeutet `x = x / 5` (teilen)
- `x %= 5` bedeutet `x = x % 5` (Rest speichern)

Zwei Fachbegriffe, die du kennen solltest:

- **Inkrementieren** heißt: einen Wert um 1 **erhöhen** – `x += 1`
- **Dekrementieren** heißt: einen Wert um 1 **verringern** – `x -= 1`

Das braucht man ständig, zum Beispiel als Zähler in einer Schleife.

So verändert `+=` den bestehenden Wert:

Beispiel

```
punkte = 10  
punkte += 5  
print(punkte)
```

Solche Zuweisungen werden **nacheinander** ausgeführt – jede arbeitet mit dem Wert, den die vorige hinterlassen hat.

Du musst das **nicht im Kopf ausrechnen**: Genau dafür hast du Python. Schreib nach jedem Schritt ein `print(x)` und schau dir an, was passiert.

Aufgabe 3: Was macht `punkte += 1`?

- ☐ a) Es erhöht `punkte` um 1 – das nennt man **inkrementieren**
- ☐ b) Es setzt `punkte` auf 1
- ☐ c) Es vergleicht `punkte` mit 1



Aufgabe 4: Starte mit `x = 13`, wende `x += 12`, dann `x *= 12`, dann zweimal `x += 1` an und gib `x` aus (soll 302 sein).

`x = 13`

Dein Code:

Vergleichsoperatoren

Mit **Vergleichsoperatoren** vergleichst du zwei Werte. Das Ergebnis ist immer ein Wahrheitswert (**True** oder **False**):

- `==` gleich · `!=` ungleich
- `<` kleiner · `>` größer
- `<=` kleiner-gleich · `>=` größer-gleich

Vergleiche liefern True/False:

Beispiel

```
print(5 == 5)
print(5 != 7)
print(7 < 5)
print(10 <= 10)
```



Der wichtigste Stolperstein: = weist zu, == vergleicht.

- `alter = 18` → speichert 18 in `alter`

- `alter == 18` → prüft, ob `alter` gleich 18 ist (ergibt `True/False`)

Wenn du in einem Vergleich aus Versehen `=` statt `==` schreibst, gibt Python einen Fehler.

Aufgabe 5: Welcher Operator vergleicht zwei Werte auf Gleichheit?

- ☐ a) `=`
- ☐ b) `==`
- ☐ c) `!=`

Aufgabe 6: Welcher Operator prüft, ob zwei Werte ungleich sind?

- ☐ a) `!=`
- ☐ b) `<>`
- ☐ c) `=/=`

Aufgabe 7: Prüfe, ob 12 ungleich 15 ist, und gib das Ergebnis aus (soll `True` sein).

Dein Code:



Logische Operatoren

Mit **logischen Operatoren** verknüpfst du Wahrheitswerte oder mehrere Bedingungen. Es gibt drei:

- **and** (logisches UND) – nur True, wenn **beide** Seiten True sind
- **or** (logisches ODER) – True, wenn **mindestens eine** Seite True ist
- **not** (logisches NICHT) – **dreht** einen Wahrheitswert um: not True → False

Mit True und False durchgespielt:

Beispiel

```
a = True
b = False
print(a and b)
print(a or b)
print(not a)
```

Aufgabe 8: Wann ist a and b True?

- ☐ a) Wenn mindestens eine Seite True ist
- ☐ b) Wenn beide Seiten True sind
- ☐ c) Immer

Bei kniffligen Ausdrücken hilft es, **von innen nach außen** zu rechnen. Beispiel mit $x1 = \text{True}$, $x2 = \text{False}$:

```
not (x1 or x2)
= not (True or False)
= not True
= False
```

Zuerst die Klammer (or), dann das not davor.

Aufgabe 9: Was ergibt not x1 and not x2 bei $x1 = \text{True}$, $x2 = \text{False}$?

- ☐ a) True
- ☐ b) False



Aufgabe 10: Gegeben `x1 = True` und `x2 = False`. Gib das Ergebnis von `x1 or x2` aus (soll True sein).

`x1 = True`
`x2 = False`

Dein Code:

Operatoren anwenden

Jetzt bringst du die Operatoren in echten Programmen zusammen. Ein klassisches Beispiel aus dem Straßenverkehr: der **Mindestabstand** zwischen zwei Autos.

Faustregel: Der Mindestabstand in Metern ist die **halbe Geschwindigkeit** (geschwindigkeit / 2).

Aufgabe 11: Frage mit `input("Geschwindigkeit: ")` die Geschwindigkeit ab, wandle sie in eine Zahl um, berechne den Mindestabstand (geschwindigkeit / 2) und gib Mindestabstand: X Meter aus.

Beispiel bei Eingabe 100: Mindestabstand: 50.0 Meter.

Dein Code:



Der **Anhalteweg** ist etwas komplexer. Er setzt sich zusammen aus:

- **Reaktionsweg** = $\text{geschwindigkeit} * 0.3$
- **Bremsweg** = $(\text{geschwindigkeit} / 10) ** 2$

Der Anhalteweg ist die **Summe** beider. Hier siehst du gleich mehrere Operatoren im Einsatz: *, /, ** und +.

Aufgabe 12: Berechne für geschwindigkeit = 100 den Anhalteweg: reaktionsweg = geschwindigkeit * 0.3, bremsweg = (geschwindigkeit / 10) ** 2, dann anhalteweg = reaktionsweg + bremsweg. Gib anhalteweg aus (soll 130.0 sein).

geschwindigkeit = 100

Dein Code: